VIDYALANKAR INSTITUTE OF TECHNOLOGY

Department of Information Technology (INFT)

Academic Year 2026-2027

A Project Report on

Turbo Apex

An Arcade Pseudo-3D Car Racing Game using Python and Pygame

Submitted in partial fulfilment of the requirements of the

Computer Graphics Lab

Submitted by

Name	**Roll No.**
Pranay Upadhyay	24101C0024
Burhanduddin Lokhandwala	24101C0013
Atharva Bhosale	24101C0008
Ashwin Agarwal	24101C0015

Department of Information Technology

Vidyalankar Institute of Technology, Wadala (E), Mumbai

2026-2027

Abstract

Turbo Apex is an arcade-style car racing game developed in Python with Pygame for a Computer Graphics Lab project. It presents a pseudo-3D road scene using perspective projection of track segments, trapezoidal road rasterization, depth ordering and layered parallax scenery. Players race around a circuit while managing speed, steering, fuel, collisions and lap progress. The game offers single-player racing against computer-controlled opponents and local two-player split-screen racing with independent player views.

Three selectable environments - Coastal Bayway, Alpine Ridge Pass and Neon Metropolis - use different track geometry, elevation, scenery and lighting palettes. Modular systems handle rendering, car physics, AI, cameras, collision, the interface and track definitions. Procedural drawing and synthesized audio reduce reliance on external assets.

Verification: the included test suite completed 14 tests with all 14 passing, using Python 3.14.2, pygame-ce 2.5.8 and SDL 2.32.10. The supplied screenshots show the main menu, single-player racing and two-player split-screen play.

Keywords

Computer graphics; pseudo-3D rendering; perspective projection; scanline rasterization; parallax; racing simulation; split-screen; collision detection; procedural graphics; Pygame.

Report Guide

SECTION	WHAT IT COVERS
1. Introduction	Motivation, problem statement, objectives and scope
2. System Design	Technology stack, architecture and runtime flow
3. Graphics and Algorithms	Projection, road rendering, sprites, depth and camera
4. Gameplay Implementation	Physics, AI, collision, fuel and race modes
5. User Interface	Menus, HUD and supplied gameplay screenshots
6. Verification and Demonstration	Executed functional checks and visible game modes
7. Limitations and Future Scope	Current constraints and planned extensions
8. Conclusion and References	Summary and technical sources

Project snapshot

PLATFORM	GAME MODES	TRACKS	VERIFICATION
Python + Pygame	Single-player; two-player split-screen	3 themed circuits	14 tests passed

1. Introduction and Motivation

Racing games are a practical setting for exploring computer graphics because they combine a moving camera, perspective, geometric transformation, depth ordering, animation and responsive input. Turbo Apex turns these concepts into a playable circuit racing experience. Although the rendered road uses a pseudo-3D technique rather than a polygonal 3D engine, perspective scaling and changing segment geometry create a convincing sense of forward motion and track curvature.

The project also demonstrates how graphics depend on systems beyond drawing: a track model supplies geometry to the renderer and AI; vehicle physics updates position and speed; collisions change motion; and the interface communicates lap, position, fuel and timing information.

1.1 Problem Statement

Design and implement a desktop racing game that renders a perspective road scene, supports keyboard-controlled racing, provides one-player and local two-player play, and communicates essential race status through an in-game HUD.

1.2 Objectives

- Implement a reusable pseudo-3D road renderer based on camera-space projection of track segments.
- Provide responsive acceleration, braking and steering with off-road behavior and lap progression.
- Support a single human driver against AI opponents and a two-player split-screen race.
- Create distinct track environments with curves, elevation, scenery and obstacles.
- Demonstrate camera modes, collision effects, fuel pickups, HUD instrumentation and menu/state transitions.
- Organize the program into maintainable Python modules and provide a verification suite.

1.3 Scope and Users

The system is a local desktop arcade game intended for students and casual players. The documented requirements are Python 3.8 or newer, Pygame CE (or compatible Pygame) and NumPy. The project is launched with `python main.py` after installing the packages in `requirements.txt`. The supplied project documentation describes the game as asset-light: much of the scenery, vehicles, interface drawing and sound is generated programmatically.

2. System Design

2.1 Technology Stack

LAYER	TECHNOLOGY	ROLE
Language	Python	Game logic, state updates, input and simulation
2D display / drawing	Pygame CE / Pygame	Window, surfaces, event queue, drawing and audio output
Numerical processing	NumPy	Support for procedural audio synthesis
Graphics approach	Custom pseudo-3D renderer	Projection, road slice drawing, sprite scaling and compositing
Project organization	Python packages	Separate game systems and map definitions

2.2 Architecture

INPUT	GAME CONTROL	SIMULATION	RENDERING
Keyboard events	State dispatcher	Cars, AI, collisions	Projected road slices
Player controllers	Race / menu states	Track and fuel	Props and vehicles
Menu selection	Pause / results	Camera and lap data	HUD and split views

The Game controller owns the active state and shared systems. State classes handle menus, map selection, racing, pause and results. Player and AI controllers update cars on the active track. The renderer consumes the track and camera state to draw a view; HUD elements then overlay telemetry and race information. In two-player mode, the renderer produces two viewports using independent player cameras.

2.3 Application Workflow

LAUNCH	MENU / MAP SELECT	RACE	PAUSE OR FINISH
Initialize Pygame and game controller	Choose single-player or two-player and a track	Update input, physics, AI, collisions, camera and HUD	Resume/restart/return, or show final standings

At each frame, the main loop processes events, updates the current state using elapsed time, and renders the resulting frame. During a race, the track position wraps around its circuit length while lap state records completed crossings. Pause and results states suspend the active driving flow and expose the corresponding menu actions.

Figure note: architecture and workflow are summarized from the supplied source layout and README. The exact call sequence is intentionally simplified for readability.

3. Graphics Concepts and Rendering

3.1 Perspective Projection

The renderer represents the road as a sequence of cross-sections along the track. For a world point relative to the camera, it computes depth and scales horizontal and vertical offsets by a perspective factor. In the implementation, points behind the camera are rejected; visible points use a scale proportional to camera depth divided by distance.

$$\Delta x = x - Cx,\ \Delta y = y - Cy,\ \Delta z = z - Cz$$
$$s = d / \Delta z$$
$$screenX = W/2 + s \cdot \Delta x \cdot W/2$$
$$screenY = H/2 - s \cdot \Delta y \cdot H/2$$
$$projectedRoadWidth = s \cdot roadWidth \cdot W/2$$

Here, (Cx, Cy, Cz) is the camera position, d is the camera depth parameter and W and H are the viewport dimensions. As Δz increases, s decreases, so distant road slices, vehicles and scenery appear smaller toward the horizon. Separate viewport dimensions allow the same projection method to serve full-screen and split-screen views.

3.2 Road Slice Rasterization

Each track segment provides projected endpoints and road edges. The renderer joins adjacent slices with filled polygons, drawing terrain, alternating curb bands, asphalt and dashed lane markings. This is a scanline-style trapezoid approach: a curved three-dimensional road is approximated by many flat bands whose screen-space boundaries shift with curvature and elevation. Track segments are drawn from far to near so nearer surfaces cover farther ones.

3.3 Depth, Occlusion and Parallax

The renderer uses painter-style depth ordering for road props, obstacles and cars. It tracks the visible road edge and uses horizon clipping to avoid drawing distant road sections through elevated terrain. Layered backgrounds scroll at different rates to suggest depth, while billboard-like scenery is scaled according to its projected distance. These choices keep the scene readable without a general-purpose 3D mesh pipeline.

3.4 Transformations and Camera

World coordinates are translated relative to the camera before projection. The car sprite is scaled with distance and can be rotated to reflect steering. The camera follows the car with smoothing, can transition between three presets (Normal, Front and Extended), and receives a brief shake effect following impacts. These operations demonstrate translation, scaling, rotation, perspective division and view-dependent rendering.

3.5 Rendering Pipeline

1. TRACK	2. PROJECT	3. DRAW ROAD	4. SCENE OBJECTS	5. HUD
Segment geometry	Camera-relative coordinates	Terrain, curbs, asphalt	Props, rivals, player car	Speed, fuel, lap, radar

4. Gameplay Systems and Implementation

4.1 Vehicle Motion and Race Progress

Cars maintain speed and lateral track position. Player input selects acceleration, braking and steering; the physics update applies acceleration, rolling deceleration, braking and stronger drag away from the racing surface. Steering and curve-related lateral forces move the car across the track. The game tracks circuit distance, lap count, current position and timing for the race HUD and results view.

4.2 Opponent AI

AI drivers use track lookahead to anticipate curves and reduce their target speed when approaching sharper bends. They monitor nearby cars and obstacles, then adjust their target lane when blocked. This combines simple heuristics with the same vehicle physics used by the player cars, keeping the opponents within the same race world.

4.3 Collision and Fuel Systems

Collision checks compare longitudinal distance and lateral separation between cars and nearby obstacles. Impacts reduce speed, apply a lateral push and may trigger a camera shake or spinout effect. Fuel decreases over time, with consumption influenced by speed. Fuel pickups restore the tank and respawn after a delay; an empty tank limits the car to a reduced speed. The HUD makes fuel level and low-fuel state visible during play.

4.4 Game Modes and Tracks

TRACK	VISUAL IDENTITY	TRACK / GAMEPLAY CHARACTER
Coastal Bayway	Bright beach, sand, palms and shoreline	Sweeping coastal turns with roadside hazards
Alpine Ridge Pass	Mountain scenery, forest and rocks	Large elevation changes and sharper bends
Neon Metropolis	Night city palette, buildings and lights	Urban corners and night-time road obstacles

4.5 Player Controls

ACTION	PLAYER 1	PLAYER 2 / SHARED
Accelerate	W	Up Arrow
Brake / reverse	S	Down Arrow
Steer left / right	A / D	Left / Right Arrow
Cycle camera	C	V or M (per project README)
Pause	Esc	Esc

In single-player mode the documented input accepts either WASD or arrow keys. In two-player mode the two key sets drive separate cars. The screenshots show the split-screen view divided into left and right player viewports.

5. User Interface and Screenshots

The interface uses a dark background with cyan, gold and red accents. Menu screens expose the available modes and help. During a race, the HUD reports lap, position and time, while the lower telemetry panel shows speed and fuel. A circuit radar provides a compact view of track layout and nearby racers. The camera mode is also displayed during play.



Figure 1. Single-player race view showing the coastal circuit, position and lap, elapsed time, speed, fuel, camera mode and circuit radar.



Figure 2. Two-player split-screen race. Each viewport identifies a player and shows its own race view and telemetry.

5. User Interface (continued)

The main menu presents the Turbo Apex title and offers Single Player, Two Players, How to Play and Exit. The control hint at the bottom explains menu navigation. This entry point keeps setup straightforward and gives players access to the rules before starting a race.

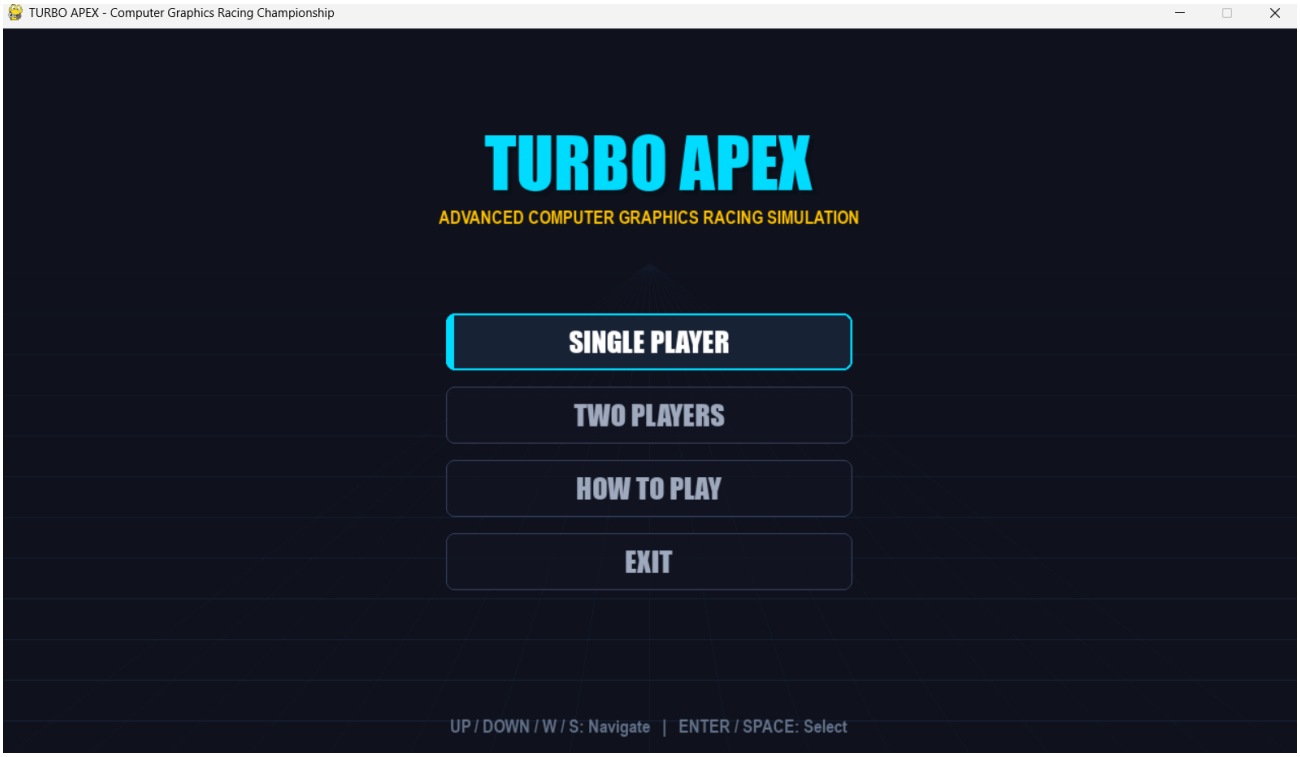


Figure 3. Main menu with the game title, mode choices and keyboard navigation instructions.

Interface Feedback

The supplied gameplay captures provide visual evidence of the visible feature set, including a countdown prompt, player identity, lap and position, speed and fuel readouts, camera label and circuit radar. They are screenshots supplied with the project request; they are presented as illustrative evidence and do not by themselves establish performance or test outcomes.

6. Verification and Demonstration

The project verification suite was executed with the included `tests_verify_all.py` file. It completed 14 tests with 14 passing, zero failures and zero errors. The run used Python 3.14.2, pygame-ce 2.5.8 and SDL 2.32.10 with the SDL dummy video driver.

Verification Results

AREA	CHECKS COVERED	RESULT
Game states	Initialization, menu navigation and map selection	Pass
Driving and multiplayer	Single-player controls; two players with separate cars and cameras	Pass
Tracks and cameras	Three map definitions and all three camera modes	Pass
Race systems	Collisions, AI driving, fuel behavior and lap progression	Pass
Session flow	Finish/results, pause, restart and return to main menu	Pass

Demonstration Summary

The supplied screenshots demonstrate the main menu, a single-player race and a two-player split-screen race. Together they show mode selection, the racing scene, player positions, lap and time indicators, speed and fuel displays, and the circuit radar. The functional run also exercised menu transitions, race start, split-screen rendering calls, pause/restart flow and results handling.

Execution summary: 14 tests run - 14 passed - 0 failures - 0 errors.

The verification confirms the tested game states and race systems. The screenshots provide a visual demonstration of the menu, single-player race and two-player split-screen mode.

7. Limitations and Future Scope

7.1 Current Limitations

The renderer is a custom pseudo-3D road renderer. Its visual vocabulary is well suited to an arcade racer, but it is not a general polygonal 3D engine. The described experience depends on keyboard input and a desktop window. The project materials do not include measured frame-rate data, device compatibility results or a user study. AI behavior is heuristic, and no online multiplayer or persistent championship progression is documented. The number of maps and camera presets is intentionally limited.

7.2 Future Enhancements

- Add controller support and configurable key bindings, with accessibility options for HUD scale and contrast.
- Introduce additional tracks, vehicle tuning and persistent best-lap records.
- Add a replay or ghost-car feature to support comparison between runs.
- Profile rendering and simulation at different window sizes, then optimize based on measured bottlenecks.
- Expand opponent behavior with richer passing strategies and difficulty settings.
- Add optional race audio and settings for volume, display and camera sensitivity.
- Document a repeatable verification procedure and record observed outcomes across supported machines.

7.3 Lessons from the Project

The project connects a simplified geometric road model to the renderer, vehicle state and AI lookahead. That shared track representation keeps simulation and display aligned. It also illustrates the value of modular organization: rendering, input, physics, camera, HUD and map data can be reasoned about separately, while the game controller coordinates their updates.

8. Conclusion

Turbo Apex is a Computer Graphics Lab racing project that brings together perspective projection, polygon rasterization, depth ordering, parallax scenery, sprite transforms and camera control in an interactive Python game. Its gameplay systems add vehicle motion, AI opponents, collisions, fuel management, lap tracking and split-screen local multiplayer. The three themed tracks and the supplied menu and race screenshots help communicate the intended arcade experience.

The repository gives a concrete implementation to study: segment projection builds the road view; modular map definitions supply distinct circuits; player and AI cars share vehicle dynamics; and the HUD makes race state visible. The included verification suite outlines a useful functional test plan. Running that suite and recording performance and compatibility measurements would be the next step toward a fully evaluated submission.

References

[1] Pygame Community Edition, “Pygame CE Documentation.” Available: https://pyga.me/docs/.

[2] Pygame Community Edition, “pygame.draw — draw geometric shapes.” Available: https://pyga.me/docs/ref/draw.html.

[3] NumPy Developers, “NumPy Documentation.” Available: https://numpy.org/doc/.

[4] D. Hearn, M. P. Baker and W. R. Carithers, Computer Graphics with OpenGL, 4th ed. Upper Saddle River, NJ: Pearson, 2011.

[5] T. Akenine-Möller, E. Haines, N. Hoffman, P. Shirley and M. Andersson, Real-Time Rendering, 4th ed. Boca Raton, FL: CRC Press, 2018.

[6] C. Ericson, Real-Time Collision Detection. San Francisco, CA: Morgan Kaufmann, 2004.

[7] Turbo Apex project source and documentation supplied with this report: README.md, game/ and maps/ modules, tests_verify_all.py, and three user-provided screenshots.

Project package summary

Entry point: `main.py` | Dependencies: `requirements.txt` | Core modules: `game/` | Track definitions: `maps/` | Verification suite: `tests_verify_all.py`